

Large-Scale and Multi-Structured Databases

Key-value Databases *Introduction*

Prof Pietro Ducange

From Arrays...

1	True
2	True
3	False
4	True
5	False
6	False
7	False
8	True
9	False
10	True

- An array is an ***ordered*** list of ***values***.
- Each ***value*** in the array is associated with an integer ***index***.
- The ***values*** are all the ***same type***

Limitations:

- The ***index*** can only be an ***integer***.
- The ***values*** must all have the ***same type***.

...to Associative Arrays...

'Pi'	3.14
'CapitalFrance'	'Paris'
17234	34468
'Foo'	'Bar'
'Start_Value'	1

- Associative arrays **generalize** the idea of an **ordered list**.
- The **index** can be a **generic identifier** such as a string, an integer and so on.
- The **values** can be of **different types**.

...to Key-Value Databases

- Data are ***persistently*** stored.
- There is ***no*** need for ***tables*** if we do not need groups of related attributes.
- The unique requirement is that ***each value*** has a ***unique identifier*** in the form of the key.
- Keys must be ***unique*** within the ***namespace*** defined in each specific ***bucket***

Bucket 1	
'Foo1'	'Bar'
'Foo2'	'Bar2'
'Foo3'	'Bar7'

Bucket 2	
'Foo1'	'Baz'
'Foo4'	'Baz3'
'Foo6'	'Baz2'

Bucket 3	
'Foo1'	'Bar7'
'Foo4'	'Baz3'
'Foo7'	'Baz9'

Essential Features of Key-Value Databases

- Simplicity
- Speed
- Scalability

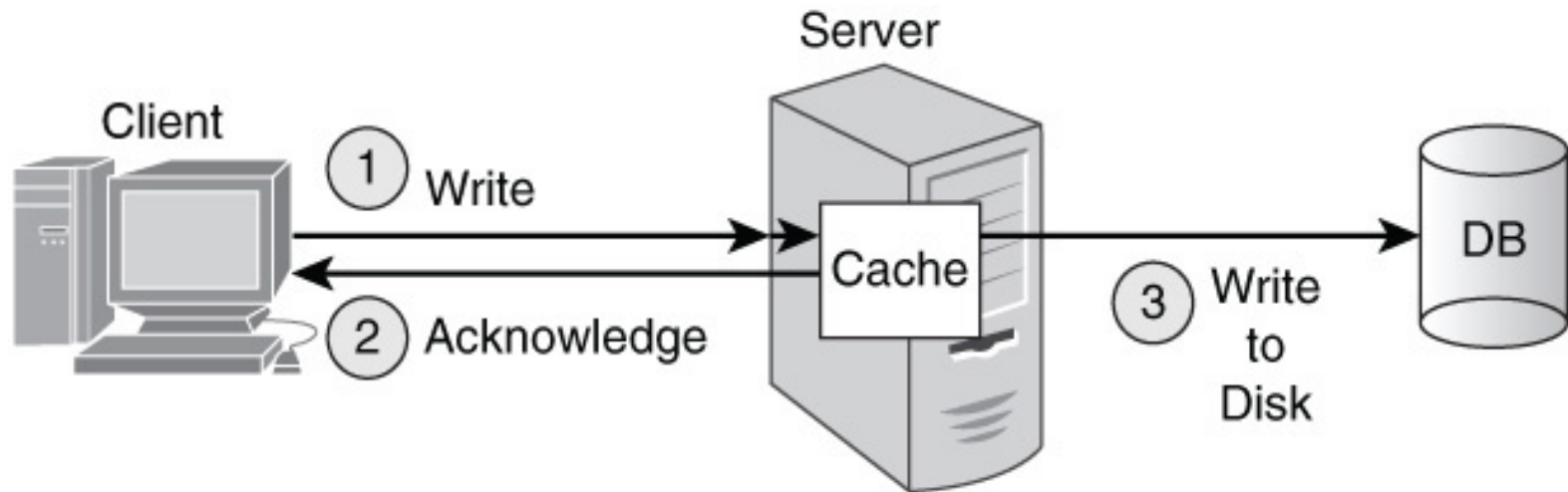
*Developers tend to use key-value databases when **ease of storage and retrieval** are **more important** than organizing data into more complex data structures, such as **tables** or **networks**.*

Simplicity

- If we do not need the **typical features** of relational databases, in particular, a **structured** and **normalized** organization of data and strong integrity **constraints**, we may exploit key-value databases.
- We do **not** have to **define** a database **schema** nor to define data **types** for each **attribute**.
- We can simply modify the code of our program if we need to handle **new attributes**, **without** the necessity of **modifying** the database.
- Key-value databases are **flexible** and **forgiving**: we can make mistakes assigning a wrong type of data to an attribute or use different type of data for a specific attribute.

Speed

Using the scheme below, the key-value database can **write** the recently updated value to disk, while the user program is doing something else.



Read operations can be **faster** if data is stored **in memory**. This is the motivation for using a **cache**: the first time the program fetches the data, it will need to read from the disk but after that the results can be saved in memory.

Scalability

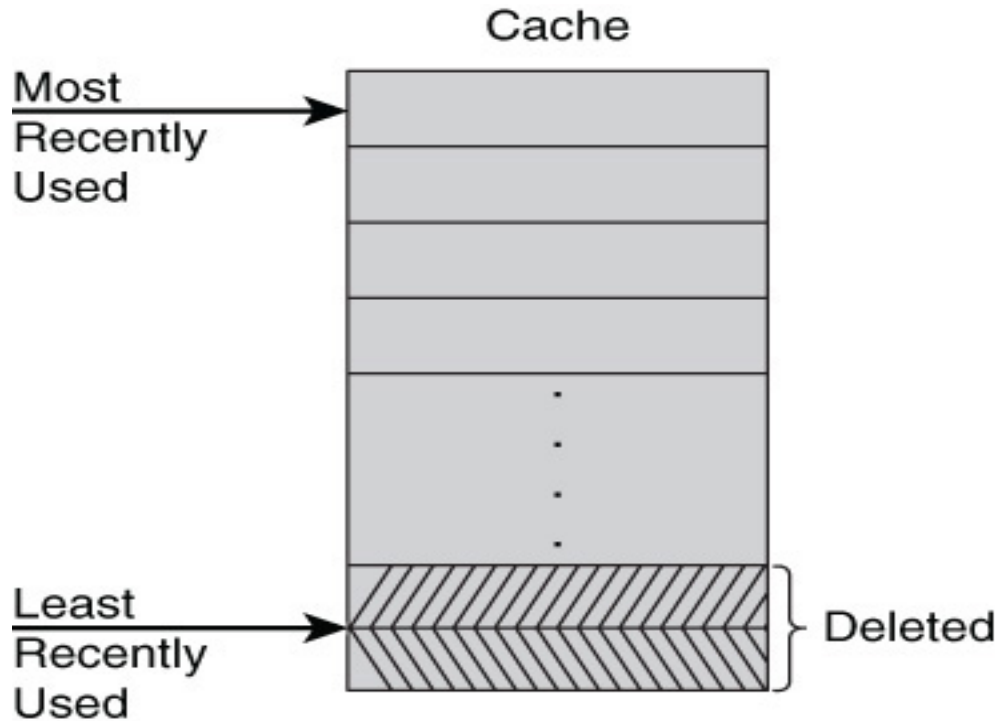
Definition: the scalability (horizontal) is the capability to **add** or **remove** servers from a cluster of servers as needed **to accommodate** the **load** on the system.

When dealing with **NoSQL databases**, employed in web and other large-scale applications, it particularly important to correctly handle both **reads** and **writes** when the system has to be scaled.

In particular, in the framework of **key-value database**, two main approaches are adopted, namely **Master-Slave Replication** and **Masterless Replication**.

The RAM is not infinite!

When the key-value database *uses all the RAM memory* allocated to it, the database will need to *free* some records.



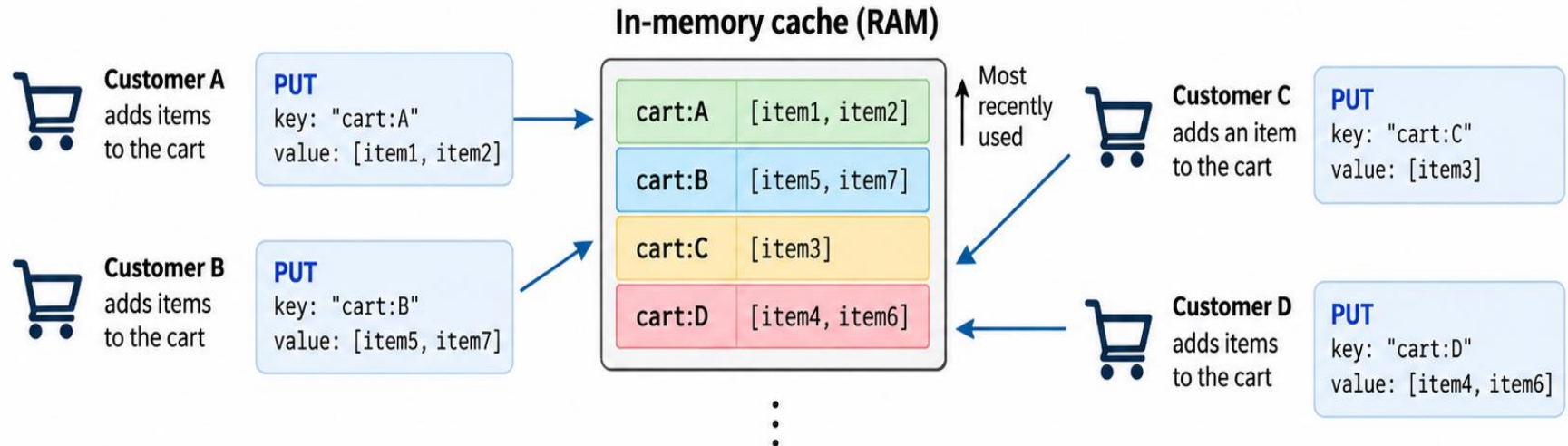
Several algorithms exist for selecting the records to remove from the RAM.

The *least recently used* (LRU) is one of the most used algorithm.

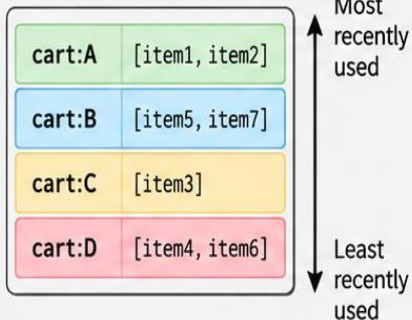
Data that has not been read or written as recently as other data can be removed from the cache.

Use case: key-value database used to store items in customers' online carts

Use case: key-value database used to store items in customers' online carts

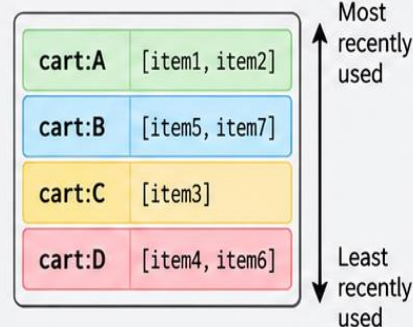


1 Cache with available space



2 Cache is full

The cache has reached the maximum RAM capacity.



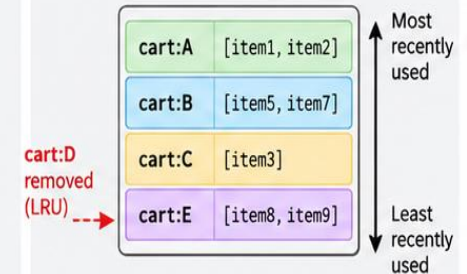
3 New item to store

Customer E adds items to the cart, but the cache is full.

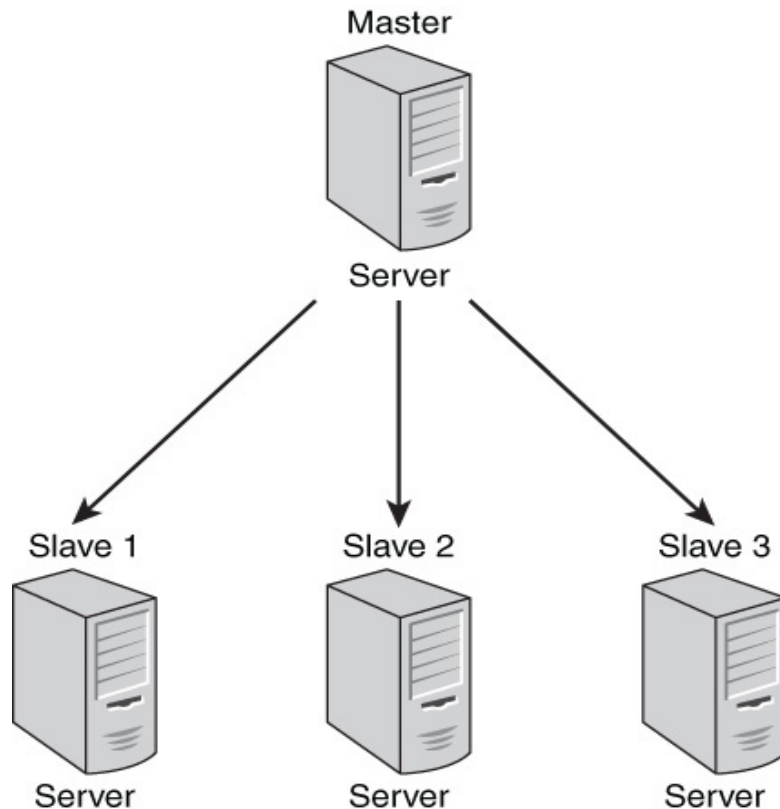
PUT
key: "cart:E"
value: [item8, item9]

4 LRU eviction

The least recently used entry (cart:D) is removed to free space, and the new entry is stored.



Master-Slave Replication



- The **master** is a server that accepts **write** and **read** requests
- The **master** is in charge of **collecting** all the **modification** of the data and **updating** data to all the slaves
- The **slaves** may only respond to **read requests**
- This architecture is particularly **suitable** for applications with a **growing demand for read operations**

Master-Slave Replication: Pros and Cons

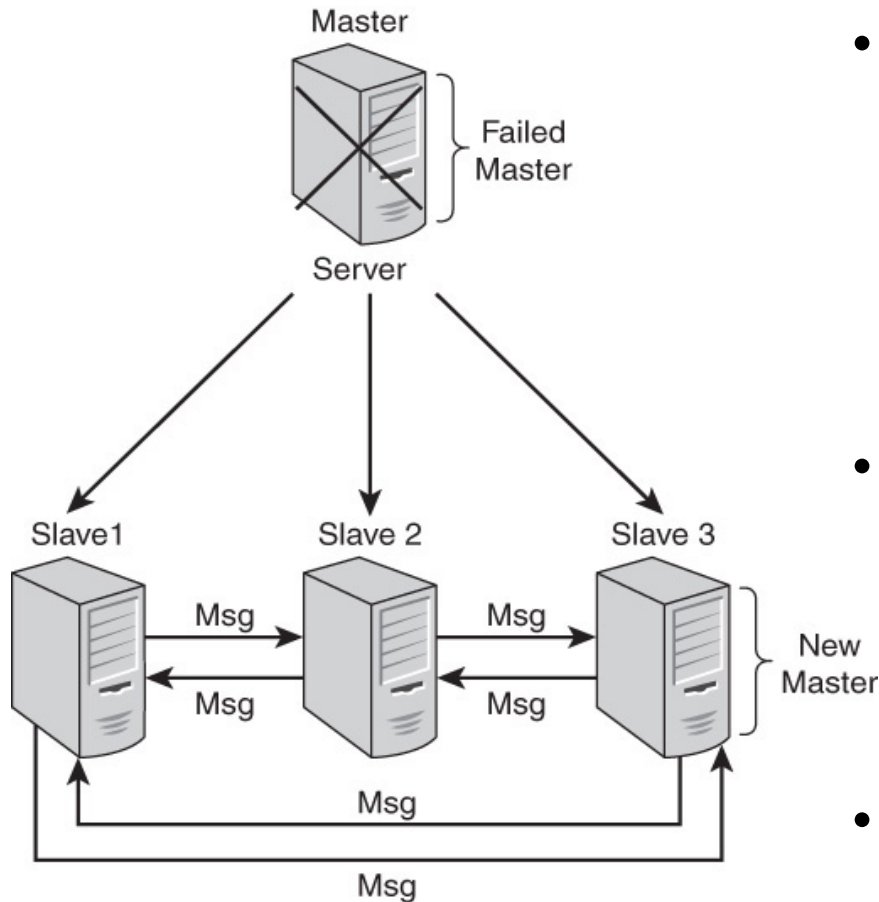
Pros:

- Very simple architecture: only the master communicates with other servers.
- There is ***no need to coordinate*** write operations or resolve conflicts between multiple servers accepting writes.

Cons:

- if the master fails, the cluster cannot accept writes.
- In this case, the entire system fails or at least loses a critical capacity, such as accepting writes.

The Higher the Complexity, the Better the Availability



- **Each** single slave **server** may send a simple **message** to ask a random server in the cluster if it is still active and if it received recently messages from the master.
- If a number of slave servers **do not receive a message** from the master within some period of time, the slaves may determine the master has failed.
- The slaves initiate a protocol to **promote** one of the slaves as a **new master**.

Masterless Replication

Use case: applications in which the number of writes may increase and achieve peaks! Example: the website which sells ticket of concerts.

The **master-slave replication** architecture **will suffer** an increasing number of write requests (only the master accept writes and all the updates must be sent to the slaves).

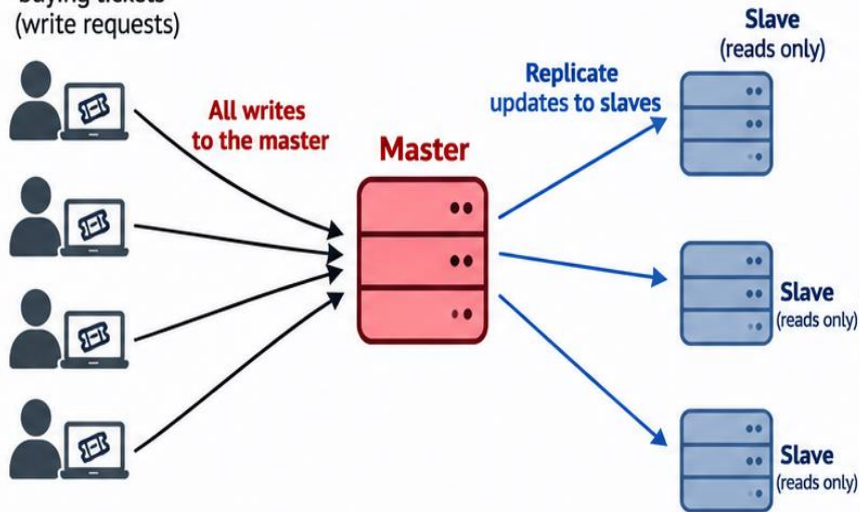
Solution: a replication model in a cluster of servers where all nodes may accept write requests.

Master-Slave Replication (problem)

Use case: concert ticket website

During popular events, the number of writes (e.g., ticket purchases) may increase and achieve peaks.

Lots of users buying tickets (write requests)



- Only the **master** accepts writes.
- All updates must be sent to the slaves.

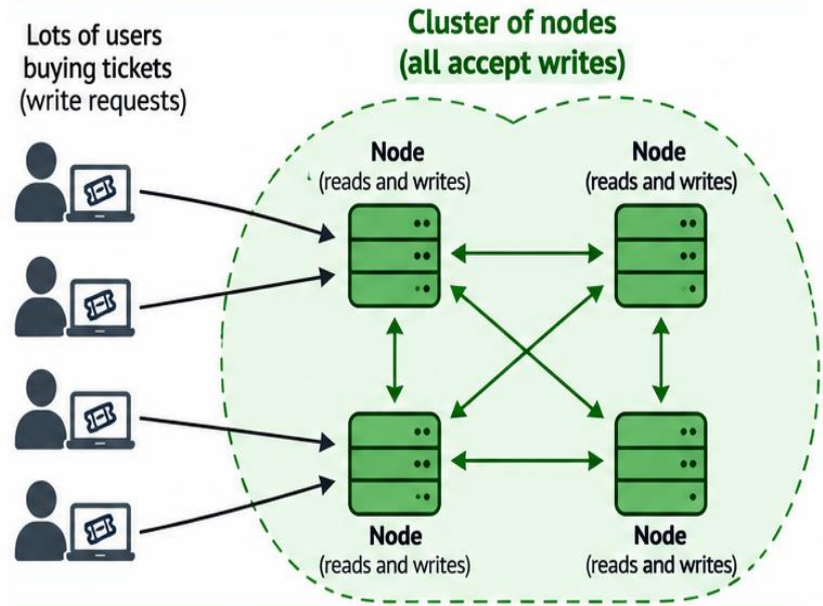
Bottleneck: the master may suffer an increasing number of write requests.

Masterless Replication (solution)

Use case: concert ticket website

During popular events, writes (e.g., ticket purchases) may increase and achieve peaks.

Lots of users buying tickets (write requests)

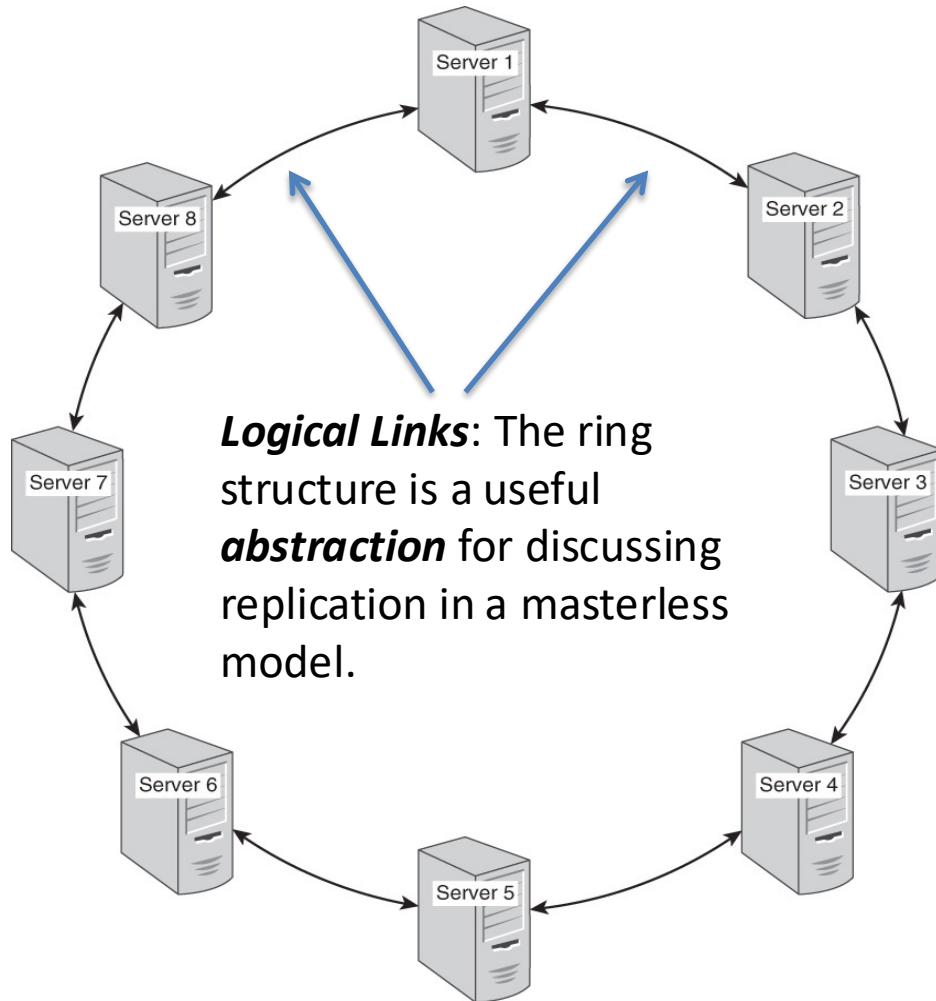


- All nodes may accept write requests.
- Updates are replicated among the nodes in the cluster (e.g., using quorum-based or gossip protocols).



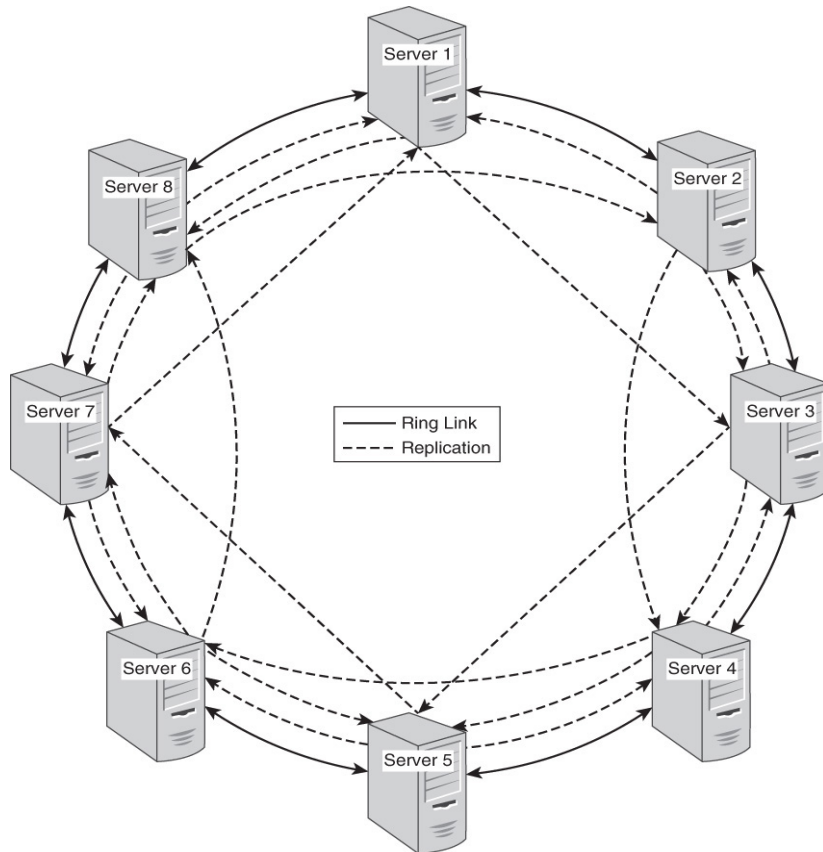
Scales writes: the write load is distributed across all nodes, avoiding a single master bottleneck.

Masterless Replication: Architecture



- There is **not** a single server that has the **master copy** of updated data.
- There is not a single server that can copy its data to all other servers.
- Each **node** is in **charge** of helping its **neighbors**.

Handling Replication: An Example



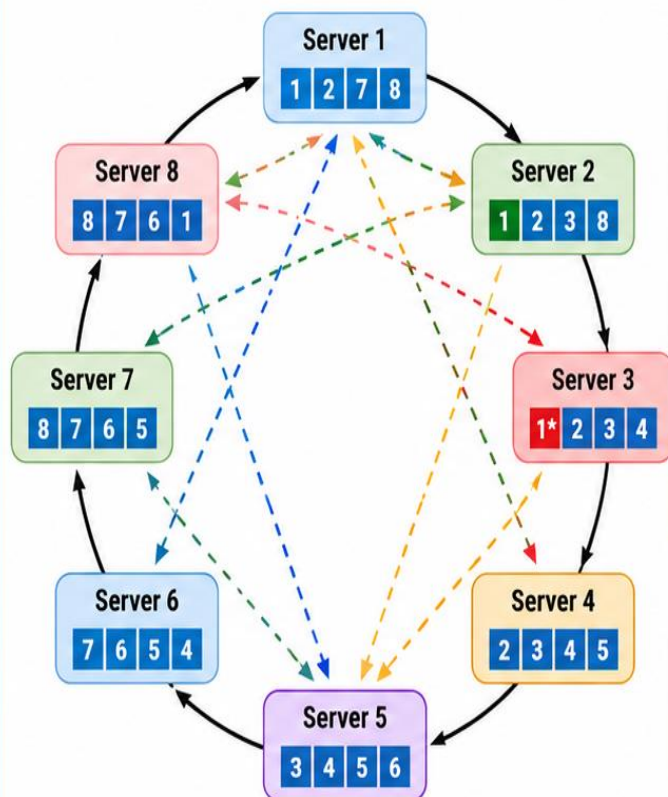
Each time there is a write operation to one of the servers, it replicates that change to the three other servers holding its replica (we consider 4 data replicas).

Each server replicates to its two neighbors and to the server two links ahead

The ring structure is an ***abstraction***. Usually, in a data center, all the servers are connected to a single network hub and are ***able to communicate with each other***.

1 Initial state

Each data item has 4 replicas: stored on the node itself, its two neighbours, and the node two links ahead in the ring.



Data placement ($r = 4$)

For each key k , the replicas are stored on:

- the node responsible for k
- its two neighbours
- the node two links ahead in the ring

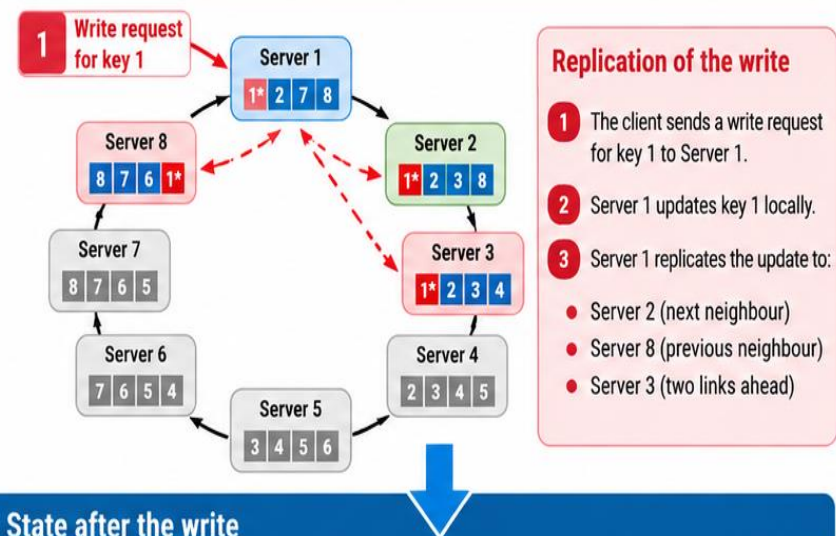
Example: key 1

Key 1 is stored on:

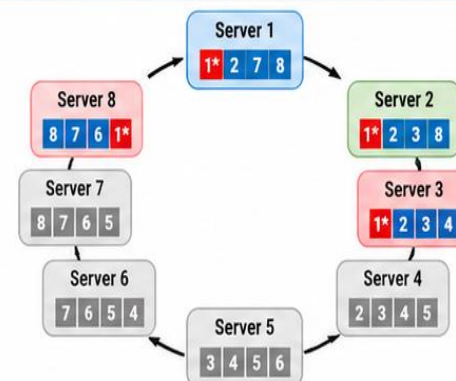
- Server 1 (itself)
- Server 2 (next neighbour)
- Server 8 (previous neighbour)
- Server 3 (two links ahead)

2 Write operation and replication

Suppose a write operation updates key 1 at Server 1. The update is replicated to its two neighbours (Servers 2 and 8) and to the server two links ahead (Server 3).



State after the write



Result

- The updated value of key 1 is stored on 4 servers.
- All other keys are unchanged.
- Future reads of key 1 can be served by any of the four replicas (Servers 1, 2, 8, or 3).

How to Construct a Key: Some Guidelines

In relational model, the ***primary key*** is used to retrieve a row in a table and it must be ***unique***.

Similarly, a ***key***, in a key-value database is used to retrieve a value and must be ***unique*** in a specific namespace.

When designing ***relational databases***, using ***meaningless*** primary keys is a good practice.

In the framework of ***key-value databases***, adopting meaningful keys is ***preferable***.

The key is the unique mean to retrieve a value!

Meaningless PK in Relational DB: An Example

1. Initial data

Surname	State	Name	BirthDate
Smith	TX	Katherine	1990-05-12
Rossi	CA	Marco	1985-03-21
Bianchi	NY	Luca	1992-11-02
Smith	CA	John	1978-07-18


**Katherine
moves from
TX to NY**

2. Katherine moves to NY

Surname	State	Name	BirthDate
Smith	NY	Katherine	1990-05-12
Rossi	CA	Marco	1985-03-21
Bianchi	NY	Luca	1992-11-02
Smith	CA	John	1978-07-18



Using (Surname, State) as primary key:

- has a specific meaning (it depends on personal data)
- can change over time



The primary key **must change** from (Smith, TX) to (Smith, NY).

This violates the principle that **primary keys are immutable**.

3. Impact on related tables

Customers

Surname	State	Name	BirthDate
Smith	TX	Katherine	1990-05-12
Rossi	CA	Marco	1985-03-21
...			

Orders (references Customers)

OrderID	Surname	State	Product	Date
	Smith	TX	Laptop	2024-01-10
1002	Rossi	CA	Tablet	2024-02-15
...				



If the key changes, we would have to **update all references** to that key in other tables (e.g., Orders, Payments, Addresses, ...).

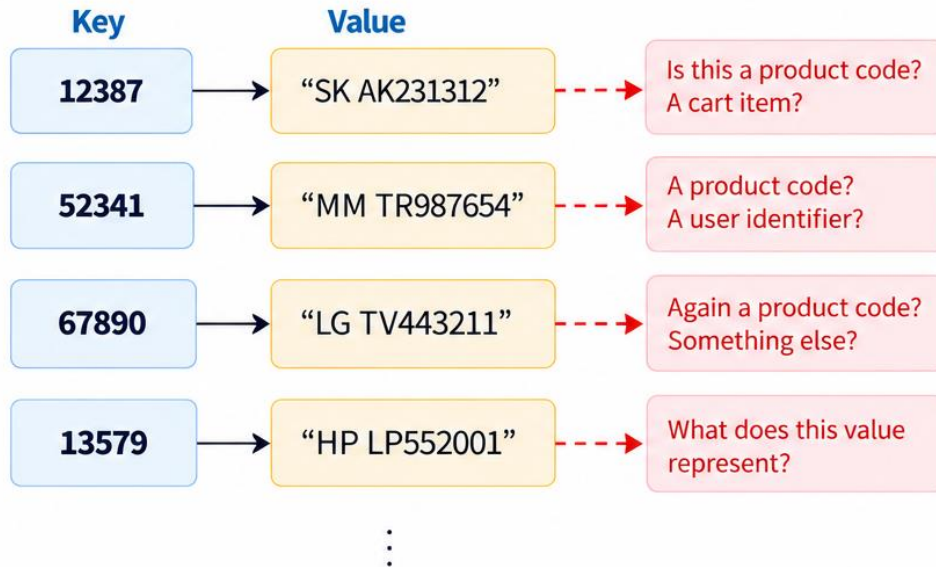


Choose primary keys without specific meanings, e.g., a surrogate key (CustomerID), which does not depend on personal data and does not change over time.

Using Meaningless Keys in a KV Database

- In a key-value database there are **no tables**, **no columns**, and **no pre-defined attributes**.
- We define a namespace, e.g., Cart, and access records using **keys**.
- If we use **meaningless keys** (just an identifier), we can store values, but **we cannot understand what each key represents**.

Namespace: Cart

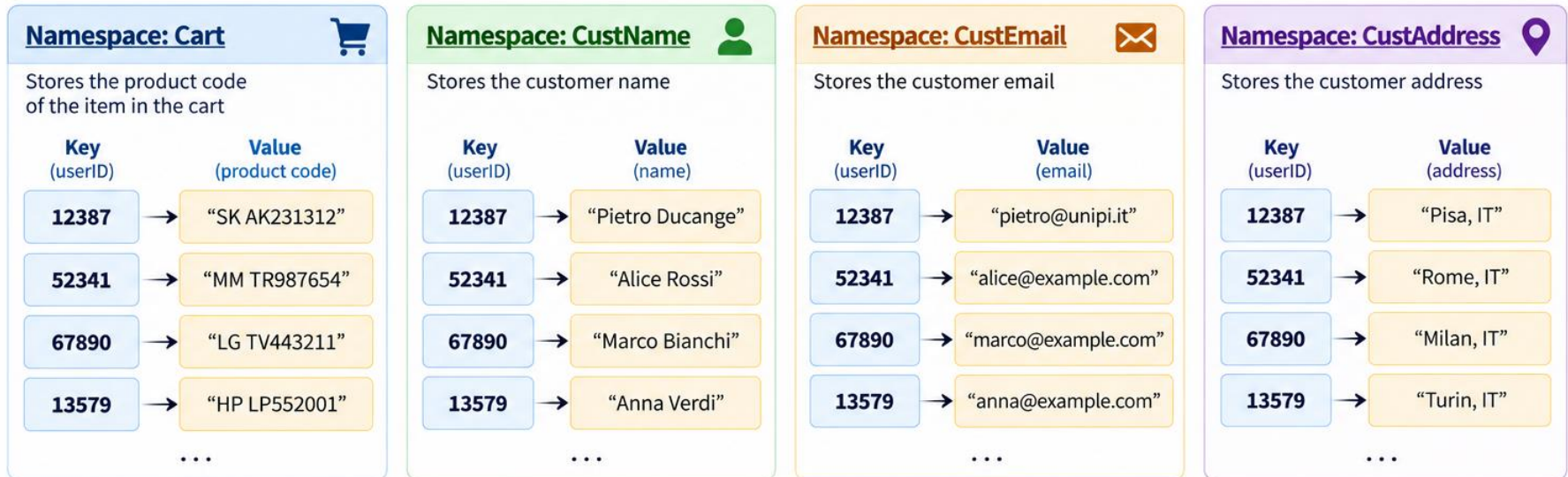


❌ Problems with meaningless keys

- Keys **do not convey any information** about the entity or the attribute.
- We **cannot understand** what each value represents (product? user? order? ...).
- To store different attributes (e.g., product code, quantity, price) we would need **different namespaces, one for each attribute**, making the design complex and similar to a relational schema.

Using Multiple Namespaces: We Go Back to a Relational Model

- To store different attributes of the same entity, we define a **separate namespace (bucket)** for each attribute.
- We access each attribute using the **same key** (the entity identifier).
- This approach reintroduces the notion of a **schema, similar to a relational model** (one table with multiple columns).



We are essentially reintroducing a relational model

- We need a **separate namespace** for each attribute.
- All namespaces share the **same key** (userID).
- This is equivalent to a **table with multiple columns** in a relational database.



Equivalent relational representation

userID	product_code	name	email	address
12387	SK AK231312	Pietro Ducange	pietro@unipi.it	Pisa, IT
52341	MM TR987654	Alice Rossi	alice@example.com	Rome, IT
67890	LG TV443211	Marco Bianchi	marco@example.com	Milan, IT
13579	HP LP552001	Anna Verdi	anna@example.com	Turin, IT
...	...	...	...	...

Meaningful Keys: One Namespace, Multiple Entities

We use a **single namespace** and **embed information in the key** to indicate the **entity**, the **entity identifier**, and the **attribute**.

Namespace: MyNamespace

Key	Value
customer:12387:firstName	"Pietro"
customer:12387:lastName	"Ducange"
customer:12387:email	"pietro@unipi.it"
customer:12387:address	"Pisa, IT"
...	...
customer:52341:firstName	"Alice"
customer:52341:email	"alice@example.com"
customer:52341:address	"Rome, IT"
...	...
product:98765:name	Concert Ticket
product:98765:description	Live performance, 2025
product:98765:price	50.00
product:98765:category	Music
...	...



Single namespace for multiple entities

All the data (e.g., customers, products, orders, ...) are stored in the **same namespace** (e.g., MyNamespace).



Self-descriptive keys

Each key clearly indicates the **entity type** (e.g., customer, product), the **entity identifier** (e.g., 12387), and the **attribute** (e.g., firstName, price, ...).



Simple and flexible design

We can easily add new entities and attributes **without creating new namespaces**.



Easy to understand and maintain

The meaning of each key is **immediately clear**, even when different entities are stored together.



Key takeaway

In a key-value database, it is important to **design keys with a meaning**, embedding the **entity name**, the **entity identifier**, and the **attribute**.

EntityName : EntityId : EntityAttribute

e.g., customer:12387:email

e.g., product:98765:price

From Key to Values

In general, **keys** may be **strings**, **numbers**, **list** or other **complex structures**.

In order to **identify a value** in the database, we actually need the “**address**” of the specific location in which this value is stored.

Hash functions are usually used to obtain the address.

The hash functions returns a number, from an **arbitrary key**.

Usually, hash functions returns values that **seem** to be **random** values.

Values returned by hash functions may be not unique (**collision problems!!**)

Hash Mapping

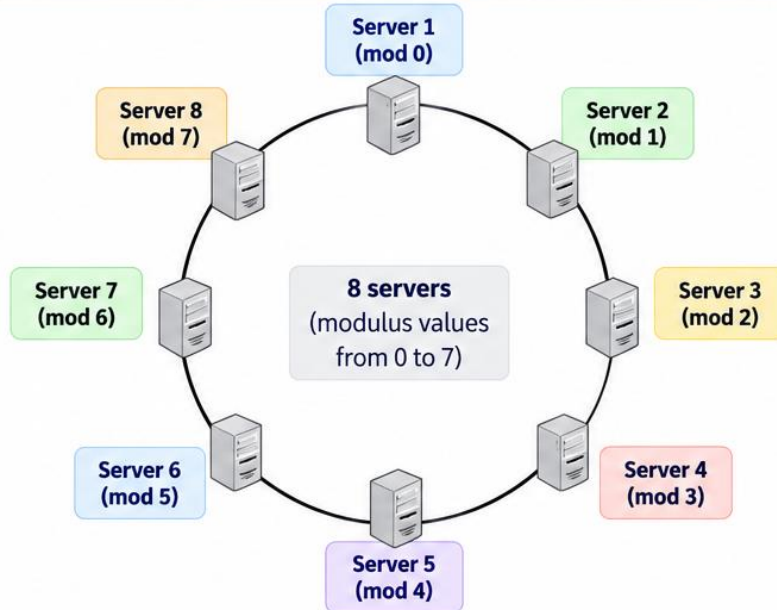
Key	Hash Value
customer:1982737: firstName	e135e850b892348a4e516cfcb385eba3bfb6d209
customer:1982737: lastName	f584667c5938571996379f256b8c82d2f5e0f62f
customer:1982737: shippingAddress	d891f26dcdb3136ea76092b1a70bc324c424ae1e
customer:1982737: shippingCity	33522192da50ea66bfc05b74d1315778b6369ec5
customer:1982737: shippingState	239ba0b4c437368ef2b16ecf58c62b5e6409722f
customer:1982737: shippingZip	814f3b2281e49941e1e7a03b223da28a8e0762ff

In the example above, the Hash Value is a number in ***hexadecimal format***.

The ***SHA-1 hash*** function has been used in the example above.

Hashing for Selecting Servers

We use a **hash function** to map each key to a server, in order to **balance the write load** among the servers in a masterless replication architecture.



1

Compute the hash of the key

We apply a hash function to the key to obtain an integer hash value.

$$h = \text{hash}(\text{key})$$

(e.g., SHA-1, MurmurHash)

2

Compute the server index

We divide the hash value by the number of servers (N) and take the remainder (modulus).

$$\text{serverIndex} = h \bmod N$$

N = 8 in this example

3

Select the server

The remainder (between 0 and 7) identifies the server responsible for the key.

$$\text{serverIndex} \in \{0, \dots, 7\}$$

Each server is assigned a number (its address).

Example: mapping keys to servers (N = 8)

Key	Hash value (h)	$h \bmod 8$	Selected server
"user:123"	834928713	1	→ Server 2 (mod 1)
"product:987"	719283451	3	→ Server 4 (mod 3)
"order:555"	205671990	6	→ Server 7 (mod 6)
"user:999"	1023456788	4	→ Server 5 (mod 4)



Key takeaway

By hashing each key and using the modulus, we obtain a simple and effective way to **distribute the write load** among the servers, without requiring a central coordinator, which fits well a masterless replication architecture.

Hashing for Selecting Servers: An Example

We use a **hash function** and the **modulus** to map each key to a specific server, so that all requests for the **same** key are handled by the **same server**.



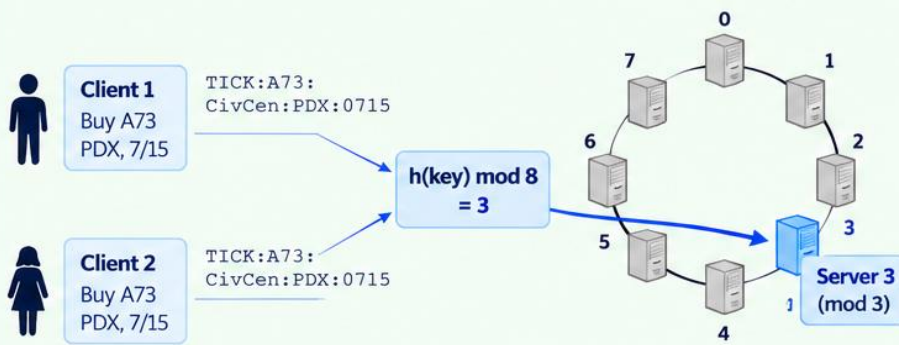
Problem: Avoid selling the same seat (A73) at the same venue (Civic Center), in the same city (Portland), on the same date (July 15) to more than one person.

Key: TICK:A73:CivCen:PDX:0715



Correct behavior: hashing + modulo

All requests for the **same key** are mapped to the **same server**.

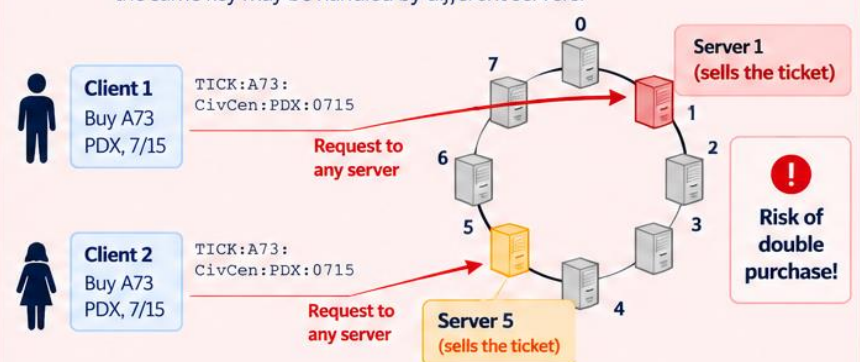


Both requests are handled by the **same server** (Server 3).
The server can coordinate the access and allow only **one successful purchase**.



Potential problem: server selected without hashing

If clients can contact **any server** (e.g., via load balancer or random selection), the same key may be handled by different servers.



The same key is handled by **different servers** (Server 1 and Server 5).
Both servers may sell the ticket, leading to a **double sale**.



Key takeaway

By hashing each key and using the modulo, we ensure that all requests for the **same resource** (e.g., the same seat at a specific venue, city and date) are routed to the **same server**, avoiding inconsistencies and enabling correct coordination in a distributed system.

Example key format

TICK:Seat:Venue:City:Date
e.g., TICK:A73:CivCen:PDX:0715

No Typing for the Values

Key-value databases do not expect us to specify types for the values we want to store.

Examples for storing the address of a customer:

- **String:** `'1232 NE River Ave, St. Louis, MO'`
- **List of strings:** `('1232 NE River Ave', 'St. Louis', 'MO')`
- **JSON format:** `{ 'Street:' : '1232 NE River Ave', 'City' : 'St. Louis', 'State' : 'MO' }`

Some Considerations

- Consider the ***design characteristics*** of the key-value database we choose to use.
- Consult the ***documentation*** for limitations on keys and values.
- When choosing a key-value database, consider the ***trade-off of various features***.
- One key-value database might offer ***ACID transactions*** but set ***limit*** on the dimension of keys and values.
- Another key-value data store might allow for large values but limit keys to numbers or strings.
- Our ***application requirements*** should be considered when weighting the ***advantages*** and ***disadvantages*** of different database systems.

Operations Allowed in Key-Value Databases

In the following, we list the operations allowed in a Key-Value database:

- To ***retrieve*** a value by key
- To ***set*** a value by key
- To ***delete*** values by key

Pay attention: If we want to do more, such as search for all addresses in which the city is “St. Louis,” we will have to do that with an application program.

Complex Queries with KV Database

```
appData[cust:9877:address] = '1232 NE River Ave, St.  
Louis, MO'
```

```
define findCustomerWithCity(p_startID, p_endID, p_City):  
    begin  
        # first, create an empty list variable to hold all  
        # addresses that match the city name  
        returnList = ();  
        # loop through a range of identifiers and build keys  
        # to look up address values then test each address  
        # using the inString function to see if the city name  
        # passed in the p_City parameter is in the address  
        # string. If it is, add it to the list of addresses  
        # to return  
        for id in p_startID to p_endID:  
            address = appData['cust:' + id + ':address'];  
            if inString(p_City, Address):  
                addToList(Address,returnList );  
        # after checking all addresses in the ranges specified  
        # by the start and end ID return the list of addresses  
        # with the specified city name.  
        return(returnList);  
    end;
```

Indexes in Key-Value Databases

Some key-value databases incorporate **search functionality** directly into the database, by maintaining an **index** on the values stored in the key-value pairs.

Key-Value Store (data)

Key	Value
cust:2149:state	IL
cust:4111:state	IL
cust:9134:state	OR
cust:7714:state	MA
cust:3412:state	MA
cust:1839:address	Boston
cust:9877:address	St. Louis
cust:1171:address	St. Louis
cust:9134:city	Portland
cust:2149:city	Chicago
cust:4111:city	Chicago
...	...

Indexing string values

For each word appearing in a value, the system keeps the list of keys that contain that word.

Inverted Index (word → list of keys)

Word	Keys
IL	cust:2149:state, cust:4111:state
OR	cust:9134:state
MA	cust:7714:state, cust:3412:state
Boston	cust:1839:address
St. Louis	cust:9877:address, cust:1171:address
Portland	cust:9134:city
Chicago	cust:2149:city, cust:4111:city
...	...

How search works

- 1 The search system **tokenizes** the value (e.g., into words).
- 2 For each word, it looks up the **list of keys** in the index.
- 3 It retrieves the corresponding **key-value pairs** from the store.



Key takeaway

A built-in search system creates an **index** that maps each word to the **list of keys** of the key-value pairs in which that word appears, enabling **efficient keyword-based retrieval**.

Suggested Readings

Chapter 3 of the book “*Dan Sullivan, NoSQL For Mere Mortals, Addison-Wesley, 2015*”

Images

All the images shown in this lecture have been extracted from:

“Dan Sullivan, NoSQL For Mere Mortals, Addison-Wesley, 2015”

Or Generated by ChatGPT under the Instructor Supervision.